**Министерство науки и высшего образования Российской Федерации**
**Федеральное государственное автономное образовательное учреждение высшего образования**
**«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»**
**(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

# Лабораторная работа № 15

## по курсу «Методы оптимизации»

«Алгоритм оптимизации роем частиц»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

*Москва 2026*

# 1 Задание

Реализовать алгоритм оптимизации роем частиц.

# 2 Результаты

Исходный код программы представлен в листинге 1.

```
using Random
using Plots

mutable struct Particle
    position::Vector{Float64}
    velocity::Vector{Float64}
    best_position::Vector{Float64}
    best_value::Float64
end

struct PSOResult
    best_position::Vector{Float64}
    best_value::Float64
    history::Vector{Float64}
    positions_history::Vector{Matrix{Float64}}
    iterations::Int
end

struct FunctionSpec
    name::String
    slug::String
    objective::Function
    lower::Vector{Float64}
    upper::Vector{Float64}
    swarm_size::Int
    iterations::Int
    grid_size::Int
    levels::Int
    velocity_scale::Float64
end

struct MethodSpec
    name::String
    variant::Symbol
    phi_p::Float64
```

```
    phi_s::Float64
    omega::Float64
    teleport_probability::Float64
    file_suffix::String
end

function schwefel(x::AbstractVector{<:Real})
    n = length(x)
    return 418.9829 * n - sum(xi * sin(sqrt(abs(xi))) for xi in x)
end

function rosenbrock(x::AbstractVector{<:Real})
    return sum(100.0 * (x[i + 1] - x[i]^2)^2 + (1.0 - x[i])^2 for i in 1:length(x) - 1)
end

function rastrigin(x::AbstractVector{<:Real})
    n = length(x)
    return 10.0 * n + sum(xi^2 - 10.0 * cos(2.0 * pi * xi) for xi in x)
end

function initialize_particle(
    objective,
    lower::Vector{Float64},
    upper::Vector{Float64},
    rng::AbstractRNG;
    velocity_scale::Float64,
)
    dimension = length(lower)
    span = upper .- lower
    position = [lower[i] + rand(rng) * span[i] for i in 1:dimension]
    velocity = [(2.0 * rand(rng) - 1.0) * velocity_scale * span[i] for i in 1:dimension]
    value = objective(position)
    return Particle(position, velocity, copy(position), value)
end

function snapshot_positions(particles::Vector{Particle})
    return reduce(vcat, (reshape(copy(particle.position), 1, :) for particle in particles))
end

function find_global_best(particles::Vector{Particle})
```

```
    best_position = copy(particles[1].best_position)
    best_value = particles[1].best_value
    for particle in particles
        if particle.best_value < best_value
            best_value = particle.best_value
            best_position = copy(particle.best_position)
        end
    end
    return best_position, best_value
end

function pso(
    objective,
    lower::AbstractVector,
    upper::AbstractVector,
    method::MethodSpec;
    swarm_size::Int,
    iterations::Int,
    seed::Int,
    velocity_scale::Float64,
)
    lower_bound = Float64.(collect(lower))
    upper_bound = Float64.(collect(upper))
    rng = MersenneTwister(seed)
    particles = [
        initialize_particle(objective, lower_bound, upper_bound, rng; velocity_scale = velocity_scale)
        for _ in 1:swarm_size
    ]

    global_best_position, global_best_value = find_global_best(particles)
    history = Float64[global_best_value]
    positions_history = Matrix{Float64}[snapshot_positions(particles)]

    for _ in 1:iterations
        for i in eachindex(particles)
            particle = particles[i]

            for d in eachindex(particle.position)
                rp = rand(rng)
                rs = rand(rng)
                base_velocity = method.variant == :inertia ? method.omega * particle.velocity[d] : particle.velocity[d]
```

```
                raw_velocity =
                    base_velocity +
                    method.phi_p * rp * (particle.best_position[d] - particle.position[d]) +
                    method.phi_s * rs * (global_best_position[d] - particle.position[d])
                particle.velocity[d] = raw_velocity
                particle.position[d] += particle.velocity[d]

                if particle.position[d] < lower_bound[d]
                    particle.position[d] = lower_bound[d]
                    particle.velocity[d] = 0.0
                elseif particle.position[d] > upper_bound[d]
                    particle.position[d] = upper_bound[d]
                    particle.velocity[d] = 0.0
                end
            end

            current_value = objective(particle.position)

            if current_value < particle.best_value
                particle.best_value = current_value
                particle.best_position = copy(particle.position)
            end

            if method.variant == :teleport && rand(rng) < method.teleport_probability
                particles[i] = initialize_particle(objective, lower_bound, upper_bound, rng; velocity_scale = velocity_scale)
            end
        end

        iteration_best_position, iteration_best_value = find_global_best(particles)
        if iteration_best_value < global_best_value
            global_best_value = iteration_best_value
            global_best_position = copy(iteration_best_position)
        end
        push!(history, global_best_value)
        push!(positions_history, snapshot_positions(particles))
    end

    return PSOResult(copy(global_best_position), global_best_value, history, positions_history, iterations)
end
```

```

function build_contour_plot(
    objective,
    lower::AbstractVector,
    upper::AbstractVector,
    result::PSOResult,
    title_text::AbstractString;
    grid_size::Int,
    levels::Int,
)
    xs = range(lower[1], upper[1], length = grid_size)
    ys = range(lower[2], upper[2], length = grid_size)
    z = [objective([x, y]) for y in ys, x in xs]

    plt = contour(
        xs,
        ys,
        z;
        levels = levels,
        color = :turbo,
        linewidth = 1.2,
        fill = false,
        aspect_ratio = :equal,
        xlabel = "x1",
        ylabel = "x2",
        title = title_text,
        legend = :outertopright,
        right_margin = 22Plots.mm,
        size = (1200, 800),
    )

    particle_count = size(result.positions_history[1], 1)

    for i in 1:particle_count
        x_track = [frame[i, 1] for frame in result.positions_history]
        y_track = [frame[i, 2] for frame in result.positions_history]
        plot!(plt, x_track, y_track; color = :black, alpha = 0.18, linewidth = 0.9, label = i == 1 ? "                    " : "")
    end

    initial_positions = result.positions_history[1]
    final_positions = result.positions_history[end]

```

```
    scatter!(
        plt,
        initial_positions[:, 1],
        initial_positions[:, 2];
        color = :gray40,
        alpha = 0.75,
        markersize = 4,
        label = "                          ",
    )

    scatter!(
        plt,
        final_positions[:, 1],
        final_positions[:, 2];
        color = :deepskyblue3,
        markersize = 5,
        label = "                        ",
    )

    scatter!(
        plt,
        [result.best_position[1]],
        [result.best_position[2]];
        color = :red,
        marker = :star5,
        markersize = 12,
        label = "                        ",
    )

    return plt
end

function build_teleport_convergence_plot(
    function_spec::FunctionSpec,
    probabilities::Vector{Float64};
    phi_p::Float64 = 0.35,
    phi_s::Float64 = 0.35,
)
    colors = [:black, :red, :green4, :orange, :dodgerblue3, :purple
    ]
    plt = plot(
        xlabel = "                  ",
        ylabel = "                                                  ",
```

```
        title = "                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
        legend = :outertopright,
        right_margin = 22Plots.mm,
        size = (1200, 800),
        linewidth = 2.2,
    )

    for (index, probability) in enumerate(probabilities)
        method = MethodSpec(
            "                                    ",
            :teleport,
            phi_p,
            phi_s,
            1.0,
            probability,
            "_teleport",
        )
        result = pso(
            function_spec.objective,
            function_spec.lower,
            function_spec.upper,
            method;
            swarm_size = function_spec.swarm_size,
            iterations = function_spec.iterations,
            seed = 42,
            velocity_scale = function_spec.velocity_scale,
        )
        label_text = string(round(probability * 100; digits = 1), "%")
        plot!(
            plt,
            0:function_spec.iterations,
            result.history;
            color = colors[index],
            label = label_text,
        )
    end

    return plt
end

function run_experiment(function_spec::FunctionSpec, method::MethodSpec)
```

```
    result = pso(
        function_spec.objective,
        function_spec.lower,
        function_spec.upper,
        method;
        swarm_size = function_spec.swarm_size,
        iterations = function_spec.iterations,
        seed = 42,
        velocity_scale = function_spec.velocity_scale,
    )

    title_text = "                $(function_spec.name), $(method.name)"
    contour_plot = build_contour_plot(
        function_spec.objective,
        function_spec.lower,
        function_spec.upper,
        result,
        title_text;
        grid_size = function_spec.grid_size,
        levels = function_spec.levels,
    )

    save_name = "$(function_spec.slug)$(method.file_suffix).png"
    savefig(contour_plot, joinpath(@__DIR__, save_name))

    println("               : ", function_spec.name)
    println("           : ", method.name)
    println("                                        : ", result.iterations)
    println("                                                  : ", result.best_position)
    println("                : ", result.best_value)
    println()
end

function run_teleport_probability_experiment(function_spec::FunctionSpec, probabilities::Vector{Float64})
    convergence_plot = build_teleport_convergence_plot(function_spec, probabilities)
    save_name = "$(function_spec.slug)_teleport_probability.png"
    savefig(convergence_plot, joinpath(@__DIR__, save_name))
end

```

```
function main()
    functions = [
        FunctionSpec("                ", "schwefel", schwefel, [-500.0, -500.0], [500.0, 500.0], 80, 250, 350, 30, 0.03),
        FunctionSpec("                      ", "rosenbrock", rosenbrock, [-2.0, -1.0], [2.0, 3.0], 60, 300, 350, 35, 0.03),
        FunctionSpec("                      ", "rastrigin", rastrigin, [-5.12, -5.12], [5.12, 5.12], 80, 250, 350, 30, 0.03),
    ]

    methods = [
        MethodSpec("                          ", :global, 0.35, 0.35, 1.0, 0, ""),
        MethodSpec("                    ", :inertia, 0.35, 0.35, 0.7, 0, "_inertia"),
        MethodSpec("                              ", :teleport, 0.35, 0.35, 1.0, 0.075, "_teleport"),
    ]
    teleport_probabilities = [0.0, 0.025, 0.051, 0.075, 0.09, 0.15]

    for function_spec in functions
        for method in methods
            run_experiment(function_spec, method)
        end
        run_teleport_probability_experiment(function_spec, teleport_probabilities)
    end
end

main()
```

Результат представлен на рисунках 1– 2.

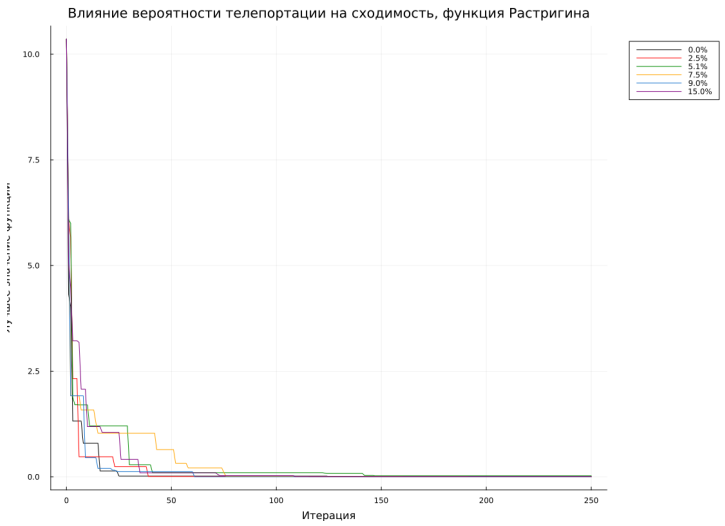


Рис. 1 — Результат

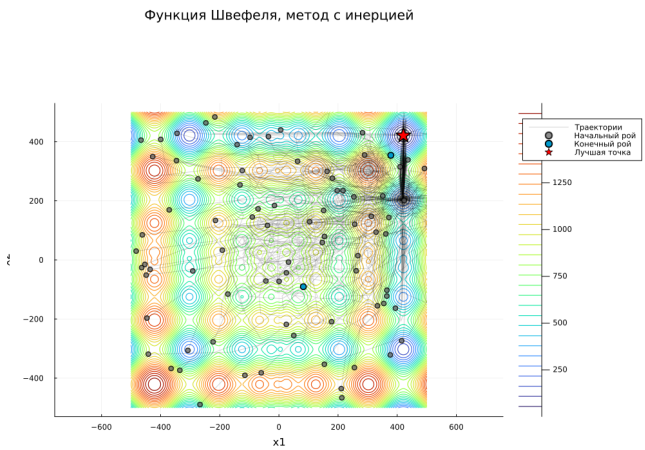


Рис. 2 — Результат